CHAPTER 26

Real-time Gaming Leaderboard

Rank millions of players by score, show the top 10 and any player's rank in real time.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a real-time leaderboard for an online game (e.g. a monthly tournament): when a player wins a match, they earn points. Show the **top 10 players**, a given player's **rank and score**, and optionally the **players around me** (relative ranking). The board resets each month (tournament).

Why interviewers ask it: Perfect Redis sorted-set question: compares a naive SQL ORDER BY solution to an in-memory ranked structure and scales it.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Which players on the board?	All players in the tournament.
Tournament period?	Monthly; leaderboard resets.
Scoring?	+1 point per match win.
Scale?	5 M DAU, 25 M MAU.
Features?	Top 10, my rank, players around me; history of past months.
Real-time?	Yes.

3. Requirements

Functional

- Update a player's score when they win.
- Display top 10.
- Show a player's rank and neighbours.
- Past-tournament leaderboards (history).

Non-functional

- Real-time updates and reads (ms).
- Scalable and available.
- Correct ranks (ties broken deterministically).
- Anti-cheat: only trusted game servers update scores.

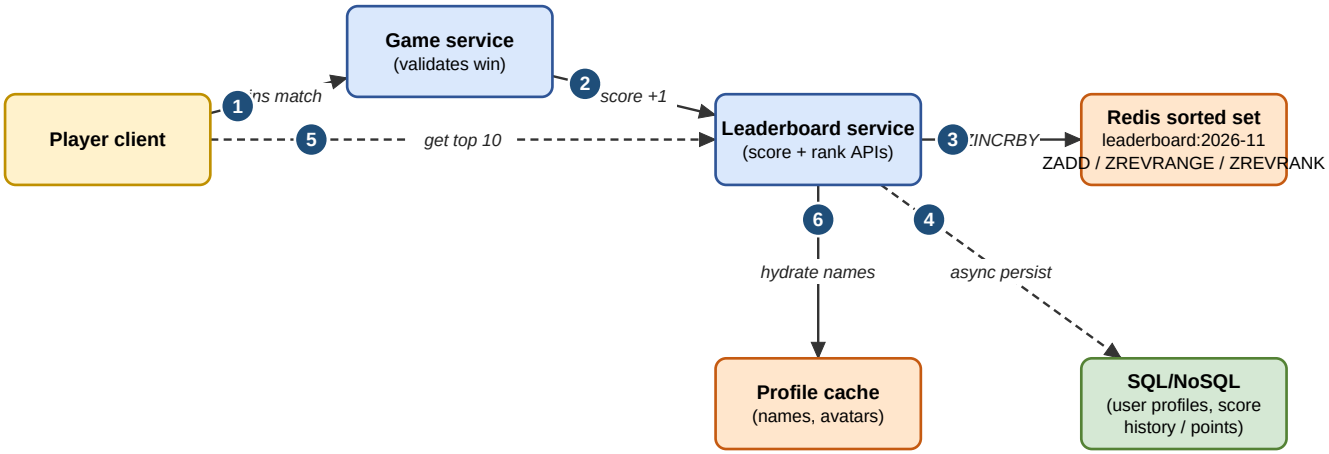
4. Back-of-the-envelope estimation

- 5 M DAU x 10 games/day = 50 M score updates/day = about **580 QPS**; peak 5x = about 2,900 QPS.
- Top-10 reads: assume each user opens leaderboard once per game -> about 580 QPS (peak ~ 3 K).
- Memory: 25 M members x (user id 24 B + score 8 B + skiplist overhead ~ 50 B) = about 2 GB in Redis sorted set. One node handles it; use replicas/sharding for growth.

5. API / data model

```
POST /v1/scores        { user_id, points }             # internal: game service only, server-to-server
GET  /v1/scores        -> top 10 [{ user_id, name, score, rank }]
GET  /v1/scores/{user_id} -> { user_id, score, rank }
```

6. High-level design



Step	What happens
1	The player wins a match, and the client reports it to the **game service**.
2	The game service **validates the result** server-side (never trust the client) and then calls the leaderboard service to add a point.
3	The leaderboard service runs `ZINCRBY leaderboard:2026-11 1 user_id` on the **Redis sorted set** (O(log N)), which updates the rank instantly.
4	Asynchronously it persists the score event to the database (the source of truth for history and recovery).
5	**Read:** the client calls `GET /v1/scores` for the top 10, or `GET /v1/scores/{user}` for its own rank.
6	The service runs `ZREVRANGE 0 9 WITHSCORES` (top 10) or `ZREVRANK` plus neighbours, then **hydrates** names and avatars from the profile cache.

7. Deep dive

Naive solution: relational DB

- `SELECT ... ORDER BY score DESC LIMIT 10` needs a scan/sort; finding a user's rank requires counting everyone with a higher score: **O(n)** and heavy under frequent updates. Even with an index, rank queries over millions of rows are too slow for real time.

Redis sorted set (the answer)

- A **sorted set** keeps members ordered by score using a **skip list + hash table**: insert/update O(log n), rank lookup O(log n), top-k O(log n + k).
- Commands: `ZINCRBY key 1 user_id` (add a point), `ZREVRANGE key 0 9 WITHSCORES` (top 10), `ZREVRANK key user_id` (my rank, 0-based), `ZSCORE key user_id`, `ZREVRANGE key (rank-4) (rank+4)` (players around me).
- Ties: Redis orders equal scores lexicographically by member; to rank by earliest achiever, encode the timestamp into the score or use composite scores.
- Keys are per tournament: `leaderboard_2026_11`; at month end archive the top results, start a new key (TTL on the old).

Storage layout

- Redis holds the live leaderboard (and could be persisted by AOF/RDB). A relational/NoSQL DB stores authoritative **score history** (`user_id, score, timestamp`) and user profiles; the leaderboard can be **rebuilt** from the DB if Redis is lost.
- Fetch names/avatars for the 10 returned ids from a user-profile cache (hydration).

Scaling: data fits, throughput may not

- With 25 M members the sorted set fits in a single node; scale **reads via replicas** and writes via vertical scaling / sharding if needed.
- **Fixed partition (sharding by score range):** e.g. shard 1: score 1-100, shard 2: 101-200, ...; top-10 reads from the highest shard; rank = (members in higher shards) + local rank. Needs a mechanism to move users between shards when score changes.
- **Hash partition:** each shard has a user subset; top-10 requires **scatter-gather** (top k from each shard and merge); rank is expensive. Fine for top-k, poor for rank. Redis Cluster does this by key hash.
- Large systems approximate rank for far-away users (percentile).

Reliability and fault tolerance

- Redis replica with failover (Sentinel/Cluster); periodically snapshot; rebuild from DB history on disaster. Queue score updates (Kafka) so Redis outages do not lose events.

Anti-cheat and security

- Only authenticated **game servers** can update scores (server-to-server auth); clients can only read. Validate game results server-side; rate-limit.

Alternative: NoSQL (DynamoDB)

- Partition key `game#month`, sort key `score`, with GSI; for rank, you still need extra structures; a global secondary index per `month_shard` helps top-k. More effort than Redis ZSET.

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Store	Relational + index / Redis sorted set	Redis ZSET: O(log n) rank; SQL only for durable history.
Sharding	Fixed score ranges / hash	Ranges give exact rank; hash simplifies writes but needs scatter-gather.
Durability	Redis only / dual-write with DB	Write-through events to DB or Kafka; rebuild if lost.
Ties	Same rank / unique ordering by time	Pick one explicitly and document.

9. Scaling, reliability, failure modes

- Read replicas for top-10 (very cacheable: short-TTL cache of top 10).
- Shard by tournament/region for natural segmentation (per-region leaderboards).
- Batch updates (pipeline) during bursts.

10. Likely follow-up questions

- How do you show 'friends leaderboard'? (per-user sorted set of friends' scores or filtered query)
- How do you handle 500 M players? (sharding by score range, approximate ranks)
- How do you reset at month end without downtime? (key rotation)
- How do you protect against cheaters?

11. Common mistakes

- Proposing a SQL ORDER BY / COUNT query for real-time rank.
- Making Redis the only copy of score history.
- Allowing clients to submit scores directly.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"The leaderboard shows the top 10 and the player's own rank, for a monthly tournament, with 5 M DAU and 25 M MAU, and a point per win. Scores must be real-time. Average QPS is small, about 50 score updates per second, with peaks near 250. Reads are the top 10 and 'my rank'."
2. High-level design	10 min	"A game service validates wins and calls the leaderboard service. The leaderboard service stores scores in a Redis sorted set and persists events to a database. Clients read the top 10 and their own rank through the leaderboard service, which hydrates profile data from a cache."
3. Deep dive	25 min	"I would first say why a relational DB fails: ranking needs a sort or count over millions of rows. A Redis sorted set gives ZINCRBY, ZREVRANGE and ZREVRANK in O(log N). One node can hold 25 M users, but for resilience I'd replicate it. If it grows to hundreds of millions, I'd shard by score range, which keeps top-N cheap, or hash-shard and merge. Each month has its own key, and old keys get archived. On a crash I rebuild from the DB."
4. Wrap up	4 min	"Extras: friends' leaderboard via a separate set, tie-breaking by earliest timestamp, around-me view, anti-cheat validation and monitoring Redis memory."

13. 60-second recap / practice checklist

- Redis sorted set: ZINCRBY / ZREVRANGE / ZREVRANK all O(log n).
- DB for history + profiles; rebuild path if Redis fails.
- Scale reads with replicas; shard by score range for rank-exactness or hash with scatter-gather for top-k.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Redis sorted sets
• Redis ZADD / ZRANK commands
• Redis: How to build a real-time leaderboard
• GeeksforGeeks: Sorted sets in Redis